NYCU-EE IC LAB – SPRING 2026

Lab04 Exercise

Design: Convolution and Deconvolution Network Accelerator

Data Preparation

1. Extract files from TA's directory:
   % tar -xvf ~iclabTA01/Lab04_2026_S.tar
2. The extracted LAB directory contains:
   a. **00_TESTBED**
   b. **01_RTL**
   c. **02_SYN**
   d. **03_GATE**
   e. **09_SUBMIT**

Design Description

In this lab, you are asked to design a **Convolution and Deconvolution Network Accelerator** that combines the feature extraction power of a convolutional neural network with the spatial reconstruction capabilities of a deconvolution network. The **convolution network** acts as a feature extractor, processing a **8x8 input image** and compressing it down to a **2x2 feature map** through two stages of **convolution**, **max pooling** and **activation function** operations. During max-pooling, it is crucial to record the exact locations of maximum activations as "**switch variables**" to preserve spatial information. The **deconvolution network** then acts as a shape generator, taking this compressed 2x2 bottleneck and reconstructing it back to a dense **8x8 output map** using **unpooling deconvolution**, and **activation function**. The unpooling operation uses the stored switch variables to place activations back into their original spatial locations, creating an enlarged but sparse map , which is subsequently densified by the deconvolution filters. By combining the semantic context captured by the convolution layers with the structural details restored by the deconvolution layers, this powerful **encoder-decoder** synergy effectively handles precise pixel-wise prediction tasks such as **semantic segmentation**.

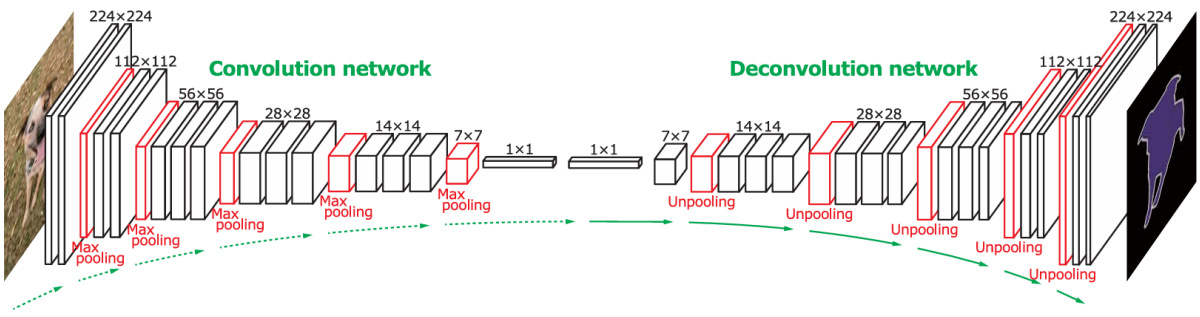


Fig 1. Convolution and Deconvolution Network Architecture Example

- **Description of input signals**

weight = kernel

The signal **instruction_in_valid** will be high for **1** cycle, then after **2~5** cycles, **weight_in_valid** will be high for **144** cycles, then after **2~5** cycles, **image_in_valid** will be high for **128** cycles.

When **instruction_in_valid** is high, the 32-bit **in_data** signal will send **instruction** for **1 cycle**. You need to decode this instruction to determine the specific mode and function to be used.

(Note that in_data[31:3] is don't care and will padding 0.)

in_data[2]	
0	Zero padding
1	Replication padding

in_data[1:0]	
00	Sigmoid
01	Tanh
10	ReLU
11	Leaky ReLU

When **weight_in_valid** is high, the 32-bit **weight** signals will continuously send data for **144 cycles**. This duration is calculated as **3** (kernel width) x **3** (kernel height) x **2** (input channels) x **2** (output channels) x **4** (total layers) = **144**. The input data is transmitted in raster scan order, as illustrated in the figure below.

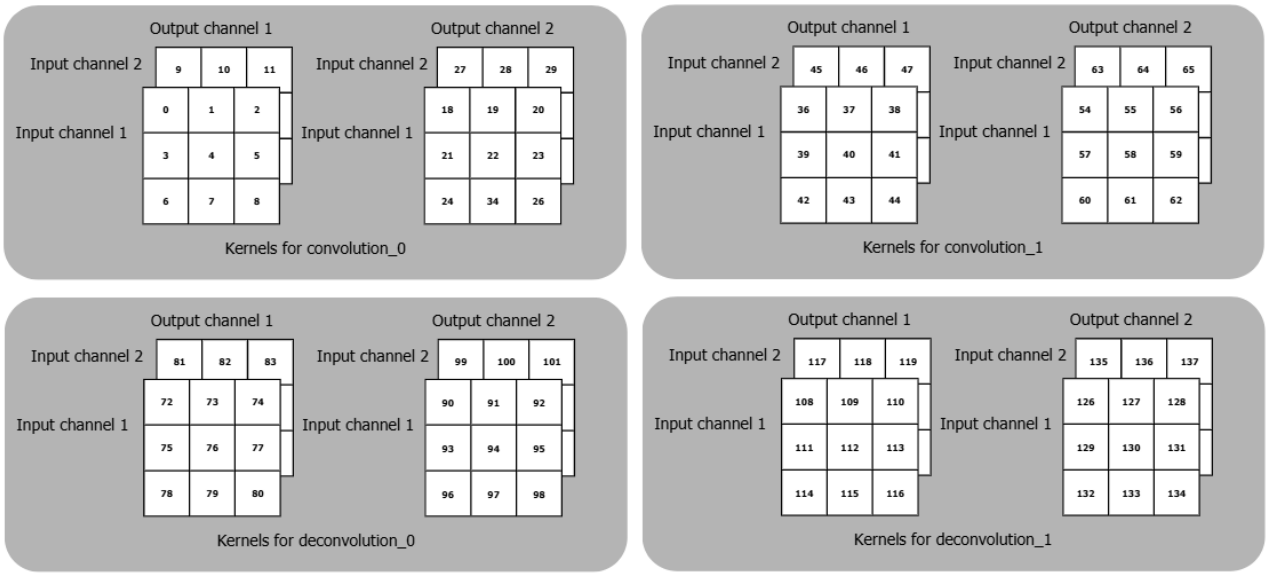


Fig 2. Input sequence of the weight signal in raster scan order

When **image_in_valid** is high, the 32-bit **image** signals will continuously send data for **128 cycles**. This duration is calculated as **8** (image width) x **8** (image height) x **2** (input channels) = **128**. The input data is transmitted in raster scan order, as illustrated in the figure below.

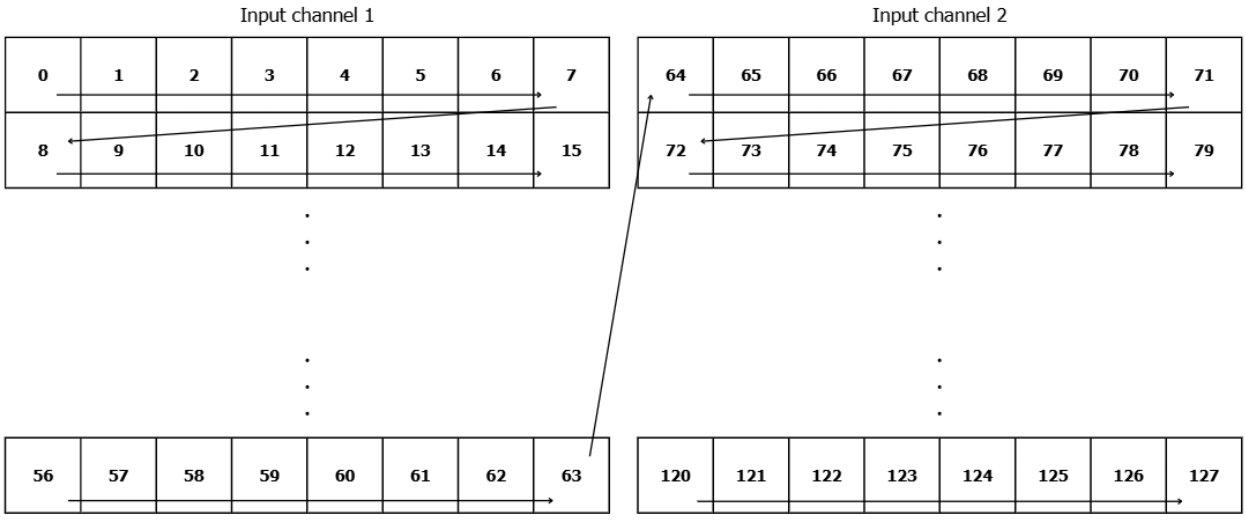


Fig 3. Input sequence of the image signal in raster scan order

- **Image Preprocessing**

After the raw image data is fed into the accelerator, directly passing unprocessed pixel values to the subsequent convolutional layers and non-linear activation functions (such as Sigmoid or Tanh) can easily cause features to fall into the 'Saturation Zone' of these functions due to excessively large or unevenly distributed values.

Therefore, in the first stage of this assignment, after the hardware receives the 8x8 input images across the two channels, it must first execute the **Min-Max Scaling** preprocessing method to complete numerical calibration.

– **Min-Max Scaling**

This method linearly compresses the pixel values of the entire image into the [0, 1] interval.

You must determine the minimum min_c and maximum max_c pixel values for each channel independently. Subsequently, for channel c, the normalized pixel value $P'_{c,i,j}$ can be computed using the formulas below:

(The test patterns for TA demo will not contain any cases where the denominator is zero.)

$$\text{For channel c: } P'_{c,i,j} = \frac{P_{c,i,j} - min_c}{max_c - min_c}$$

- **Overall Architecture of CNN**

After the preprocessing, we can implement our CNN now. This diagram illustrates the feature extraction process, sequentially applying padding, convolution, max-pooling, and activation operations to compress the 8x8 input image into a 2x2 bottleneck feature map.

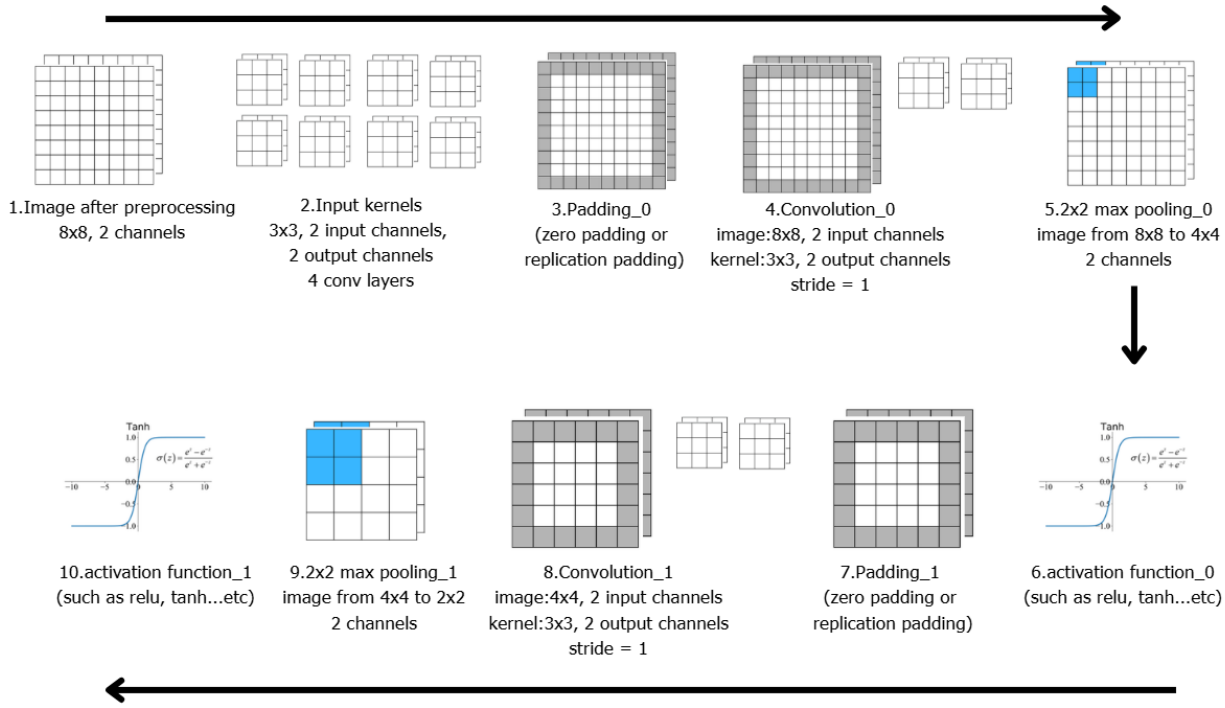


Fig 4. Data flow of the Convolution Network (Encoder stage).

This diagram demonstrates the spatial reconstruction process, utilizing unpooling, padding, and deconvolution layers to expand the 2x2 bottleneck feature map back into an 8x8 dense output map.

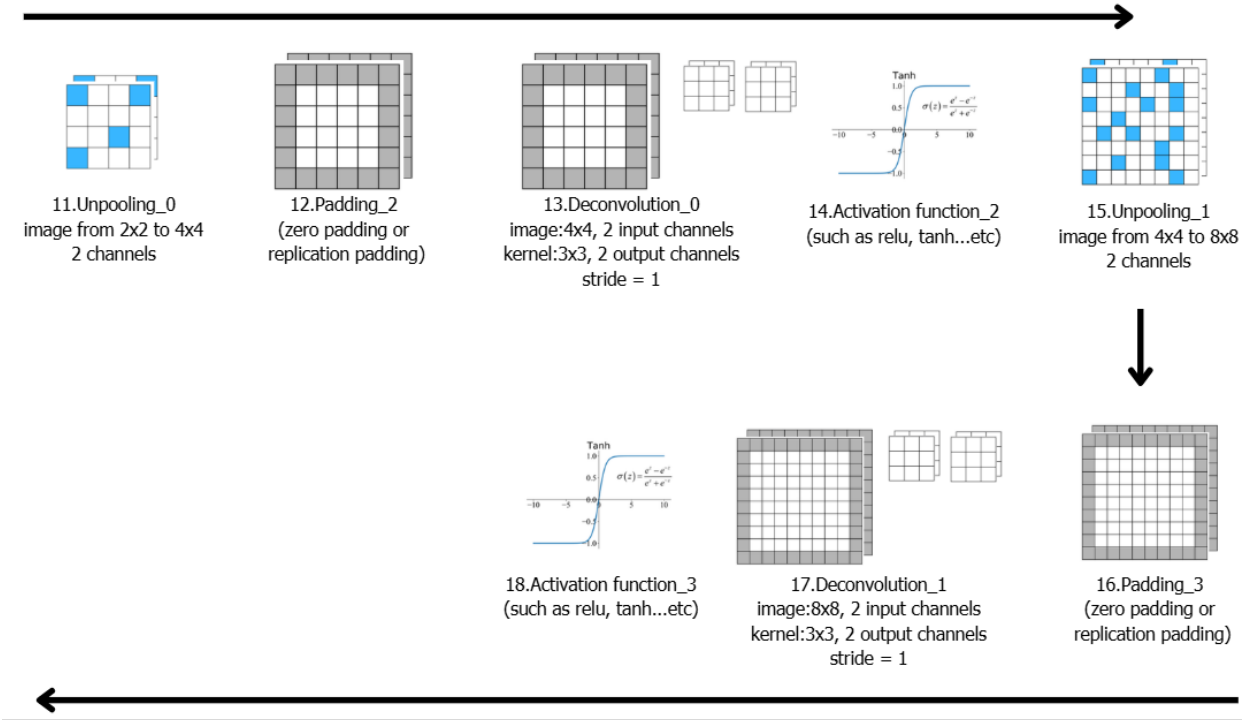


Fig 5. Data flow of the Deconvolution Network (Decoder stage).

- **Description of CNN sub-networks**

Before doing the convolution, you must perform **Replication Padding or Zero Padding** according to the information given by **instruction**. The padding width is 1.

- **Replication Padding**

  Replication padding, is a technique used in image processing and computer vision to extend the borders of an image by replicating or mirroring the existing pixels.

- **Zero Padding**

  Zero padding involves adding zeros around the borders of an image or signal before applying certain operations, such as convolutions or Fourier transforms.

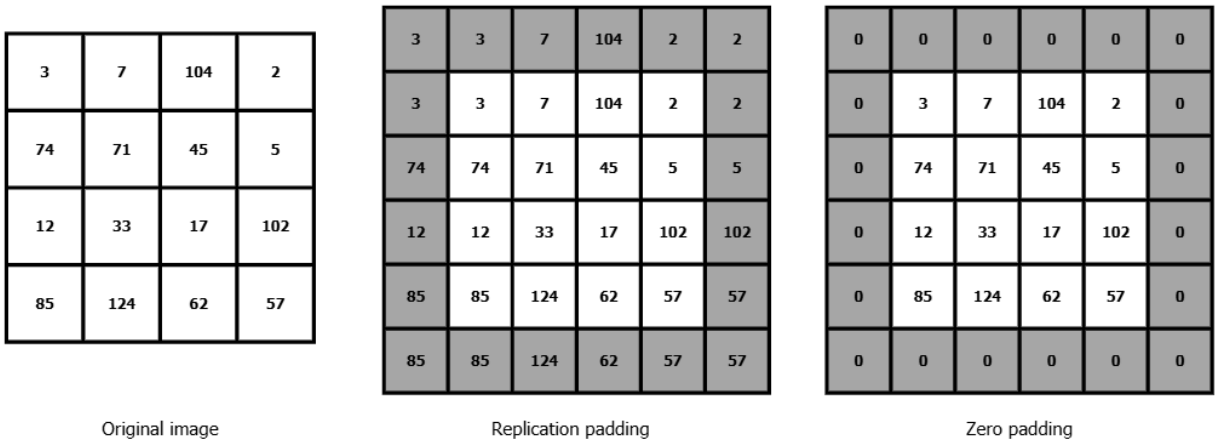


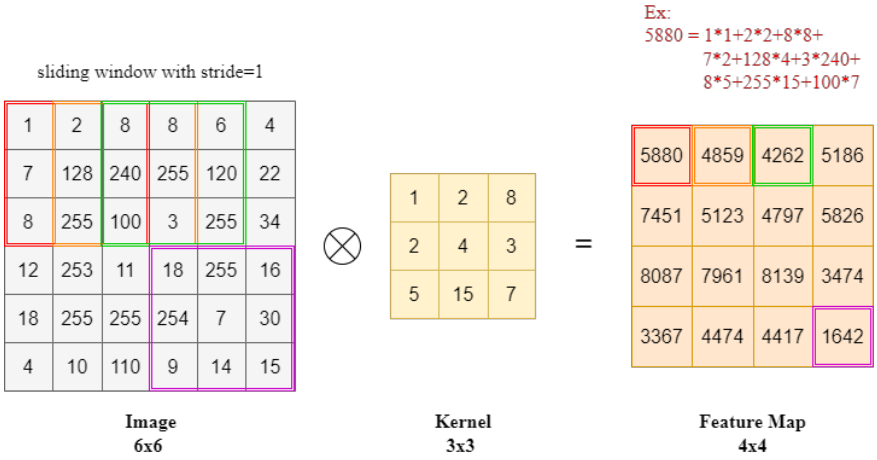
Fig 6. Replication padding and Zero Padding

- **Convolution/Deconolution**

Hint: Convolution and deconvolution stage use the exact same calculation formula and method.

Formula of convolution:

$$FeatureMap[\mathrm{m},\mathrm{n}] = \sum_{j}\sum_{i} Image[m,n] \cdot Kernel[m-i, n-j]$$



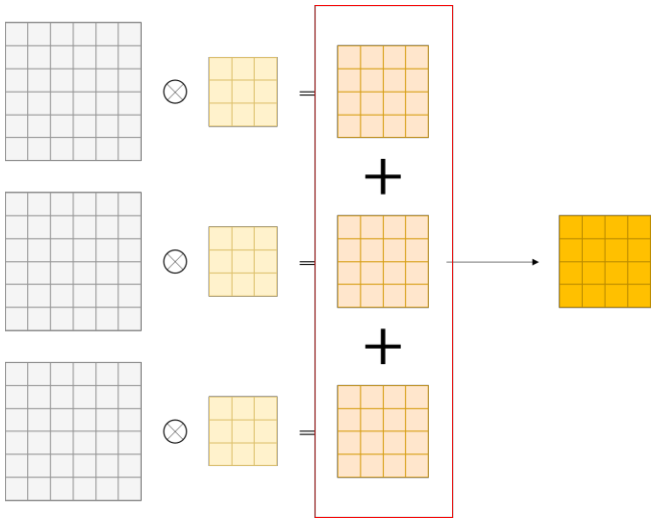


Fig 7. Example of convolution operation

- **Max Pooling**

The max pooling operation works by sliding a 2x2 window, over the input feature map and taking the maximum value in each window as output. This operation is only used at convolution.

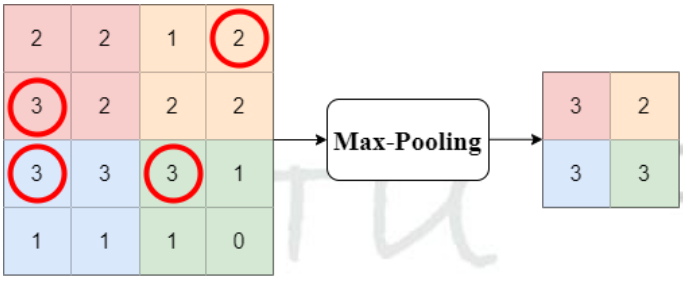


Fig 8. Example of max pooling operation

- **Unpooling**

During the 2x2 max-pooling operation in the convolution network, you must record the exact spatial locations of the maximum activations. This position information is later reused during the unpooling operation. For instance, as illustrated in Fig 9, if the maximum value in a pooling window is located at the top-right corner, you must store this specific position. During the subsequent unpooling stage, you will place the value back into that exact top-right corner of the 2x2 grid, and strictly fill the remaining three positions with zeros. This operation is only used at deconvolution.

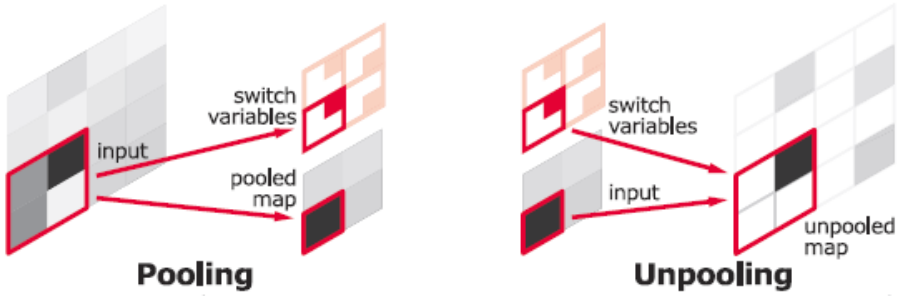


Fig 9. Max pooling and unpooling

To clarify the symmetric relationship between the encoder and decoder stages, the mapping of spatial position records (switch variables) is summarized in the table below. As for the name of the stages, you can refer to **Fig 4** and **Fig 5**.

Deconvolution stage	Input image size	Output image size	Position information
Unpooling_0 Channel 0	2x2	4x4	Max pooling_0 Channel 0
Unpooling_0 Channel 1	2x2	4x4	Max pooling_0 Channel 1
Unpooling_1 Channel 0	4x4	8x8	Max pooling_1 Channel 0
Unpooling_1 Channel 1	4x4	8x8	Max pooling_1 Channel 1

- **Activation Function**

An activation applies a non-linear transformation to the result of CNN. you must perform **Sigmoid** or **tanh** or **ReLU** or **leaky ReLU** according to the information given by **instruction**.

For leaky ReLU, a = 0.125.

For leaky ReLU, if -2^-123 < x ≤ 0, f(x) = 0.

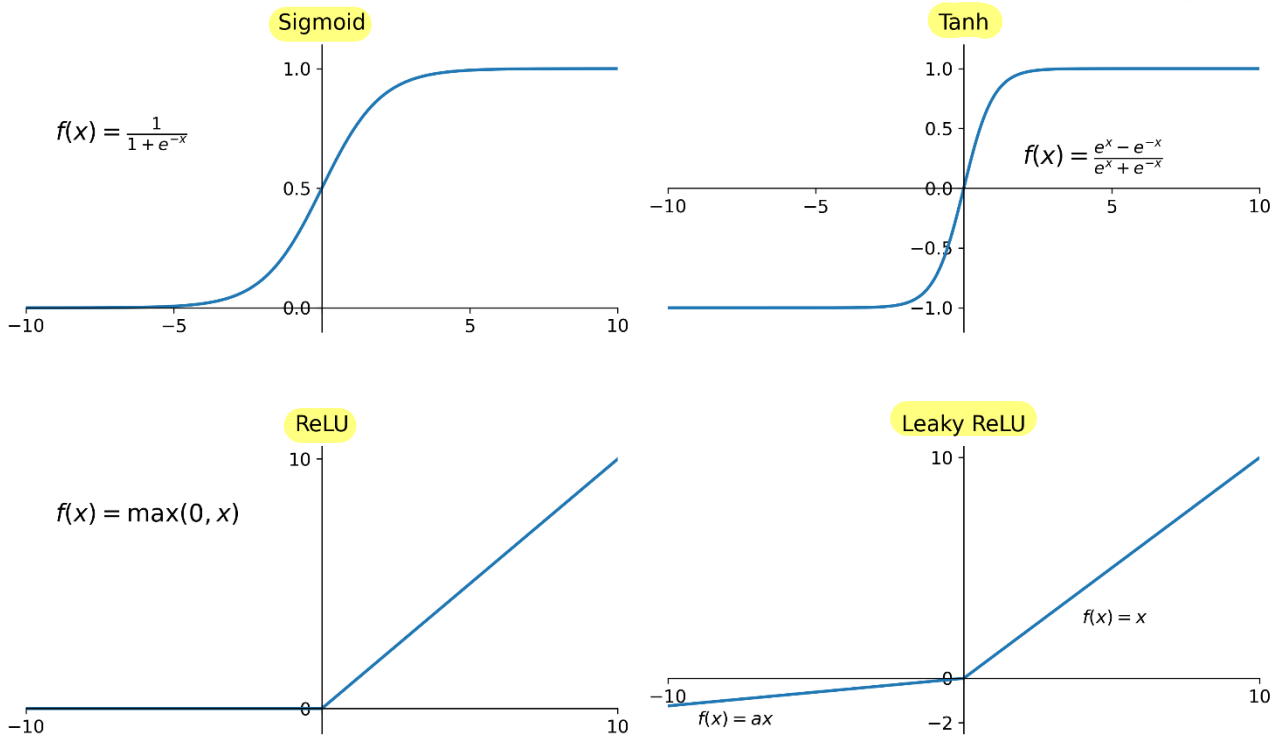


Fig 10. Activation function

- Description of output signals

Similar as input image, the **out_valid** must be pulled high for 128 consecutive cycles. You must let **out_data** transmitted in raster scan order, as illustrated in the figure below. The arithmetic representation of **out_data** must follow the IEEE-754 floating number format.

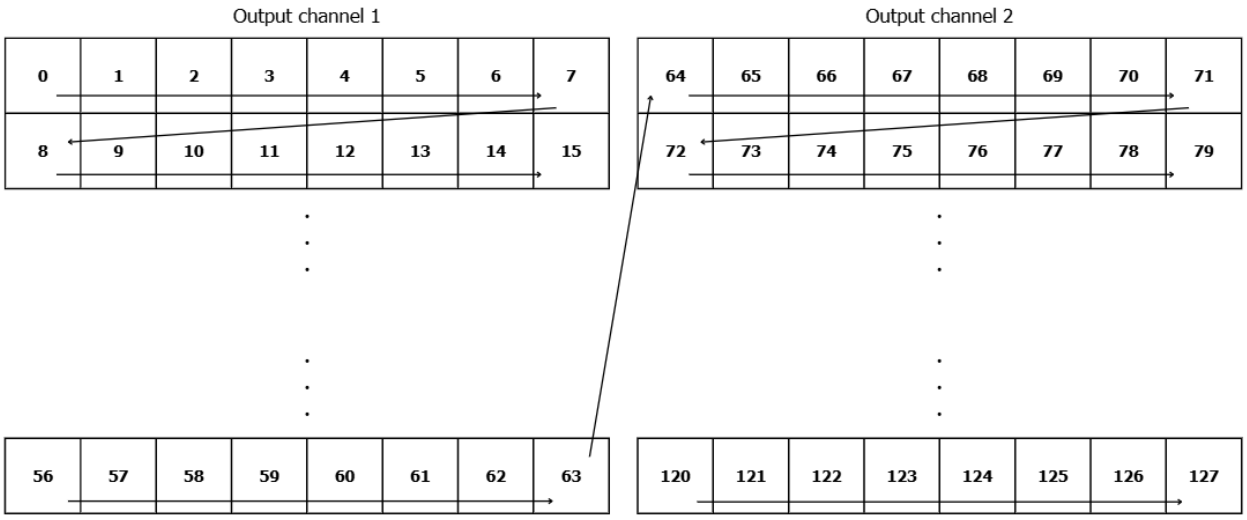


Fig 11. The order of output data

Inputs and Outputs

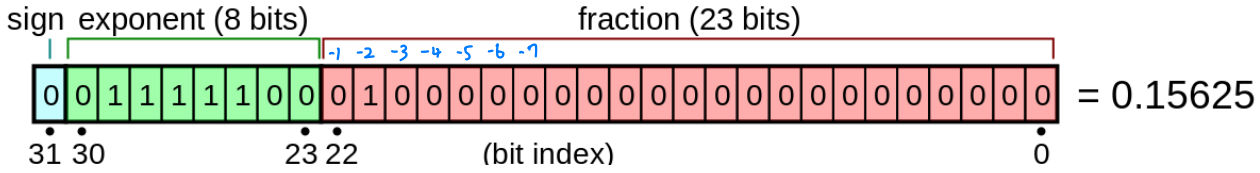
The following are the definitions of input signals

Input Signals	Bit Width	Definition
clk	1	Clock.
rst_n	1	Asynchronous active-low reset.
instruction_in_valid	1	High when “in_data” is sending instruction values.
image_in_valid	1	High when “in_data” is sending image values.(Range: -127~128)
weight_in_valid	1	High when “in_data” is sending weight values.(Range: -1~1)
in_data	32	Input data. When “instruction_in_valid” is high, in_data is sending instruction value. When “image_in_valid” is high, in_data is sending image value. When “weight_in_valid” is high, in_data is sending weight value.

The following are the definitions of output signals

Output Signals	Bit Width	Definition
out_valid	1	Pull high when you finish your calculation. Must be pulled high for 128 consecutive cycles.
out_data	32	The output signals of image. The arithmetic representation follows the IEEE-754 floating number format.

Each time you output the result, our pattern will check the correctness of it. Basically, if you follow the formulas and use IEEE floating point number IP, you should get same result as our answer. **However, we release the constraint; you may have an error under 0.002 for the result after converting to float number. This means that we will convert your output from binary format into real float number, and compare with our answer. Error will be calculated by '(golden-ans)/golden' and get its absolute value. If the error is higher than the value, you will fail this lab.**



$$\text{sign} = +1$$
$$\text{exponent} = (-127) + 124 = -3$$
$$\text{fraction} = 1 + 2^{-2} = 1.25$$
$$\text{value} = (+1) \times 1.25 \times 2^{-3} = +0.15625$$

Binary form: 00111101101100101010000001000101
IEEE floating number: 0.08721975
nWave:



nWave will round the number for display, but the computation will not be affected. The following numbers are all from nWave, thus the computations are performed by IPs in Verilog.
※ e-02 = 10^{-2}

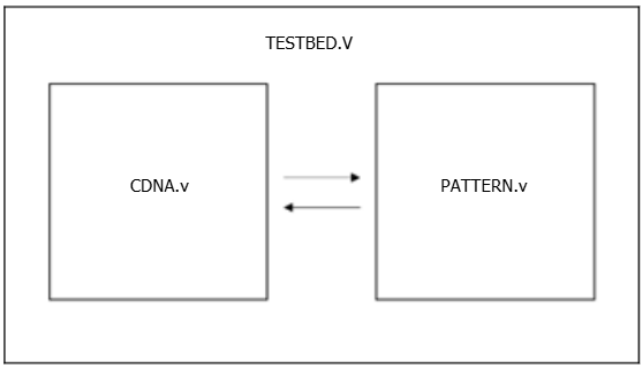
1. The input signal **instruction_in_valid** is high for **1 cycle**. And **in_data** is delivered instruction.
2. The input signal **weight_in_valid** is high for **144 cycles** continuously. And **in_data** is delivered weights in raster scan order.

3. The input signal **image_in_valid** is high for **128 cycles** continuously. And **in_data** is delivered images in raster scan order.
4. When both **instruction_in_valid, image_in_valid and weight_in_valid** are low, **in_data** is tied to unknown state.
5. All input signals are synchronized at negative edge of the clock.
6. The output signal **out_data** must be delivered for **128 cycle** continuously, and **out_valid** should be **high** simultaneously.
7. The **out_data** signal should be **zero** when **out_valid** is low.
8. The **out_valid** cannot overlap with **in_valid** at any time.
9. **You can modify the parameters of DW such as arch. However, you must not change the IEEE 754 single-precision format parameters (i.e., the 8-bit exponent and 23-bit fraction). Or you might fail the demo.**
10. **You don't need to worry about infinity in the calculation process.**

Specifications

1. Top module name: CDNA (File name: CDNA.v)
2. **You have to check an error under 0.002 for the result after converting to float number. If the error is higher than the value, you will fail this lab.**
3. **It is asynchronous reset and active-low architecture. If you use synchronous reset (considering reset after clock starting) in your design, you may fail to reset signals.**
4. The reset signal (rst_n) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
5. The **out_data** should be reset after your **out_valid** is pulled down.
6. The execution latency is limited in **1200 cycles**. The latency is the clock cycles between the falling edge of the **image_in_valid** and the rising edge of the first **out_valid**.
7. The area is limited in **10,000,000**. Also, the synthesis time should be less than **4 hours**.
8. You can adjust your clock period by yourself, but the maximum period is **50 ns**. The precision of clock period is 0.1, for example, 4.5 is allowed, 4.55 is not allowed.
9. The input delay is set to **0.5*(clock period)**.
10. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.
11. The synthesis result of data type **cannot** include any **latches**.
12. After synthesis, you can check CDNA.area and CDNA.timing. The area report is valid when the slack in the end of timing report should be **non-negative (MET)**.
13. **In this lab, you must use at least one IEEE floating point number IP from Designware. We will check it at CDNA.resource in 02_SYN/Report/.**

Block Diagram



Grading Policy

1. Function Validity: 70%
2. Performance: 30 %
   - Area * (Computation time)^2: 30%
   - Computation time = Latency * Clock period
3. Performance Comparison (1de vs. 2de):
   - The performance of 1de and 2de submissions will be evaluated and ranked separately.
   - The total score for 1de submissions will range from 70 to 100.
   - The total score for 2de submissions will range from 49 to 70.

Note

1. Please submit following files under 09_SUBMIT before 12:00 at noon on March. 30:

- CDNA.v
- If uploaded files violate the naming rule, you will get 5 deduct points.
- In this lab, you can adjust your clock cycle time.

- The 2nd demo deadline is **12:00 at noon on April.1** .
- Check whether there is any wire / reg / submodule name called “error”, “fail”, “pass”, “congratulation”, “latch”, “DW_fp”, if you used, you will fail the lab.

2. Template folders and reference commands:

01_RTL/	(RTL simulation)	**./01_run_vcs_rtl**
02_SYN/	(Synthesis)	**./01_run_dc_shell**
	(Check if there is any **latch** in your design in **syn.log)**	
	(Check the timing of design in /Report/ CDNA**.timing**)	
03_GATE /	(Gate-level simulation)	**./01_run_vcs_gate**
09_SBMIT /	(submit your files)	**./00_tar**
		./01_submit
		./02_check

※You should make sure the two clock period values identical in 00_TESTBED/PATTERN.v && /02_SYN/syn.tcl

3. **The demo patterns used by the TAs will be released on e3 at noon on March.31.**

Sample Waveform

```
`define CYCLE_TIME      50.0
`define SEED_NUMBER     28825252
`define PATTERN_NUMBER 1000
```

```
#=========================
# (A) Global Parameters
#=========================
set DESIGN "CDNA"
set CYCLE 50
```



Fig1. Input waveform (CLK will be active 2~5 clk periods after reset)

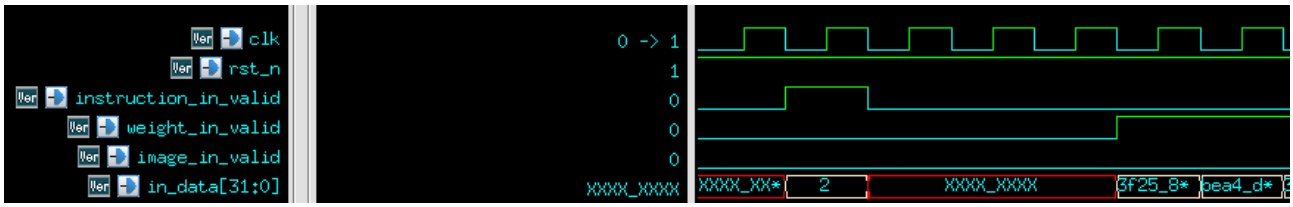


Fig2. Input waveform

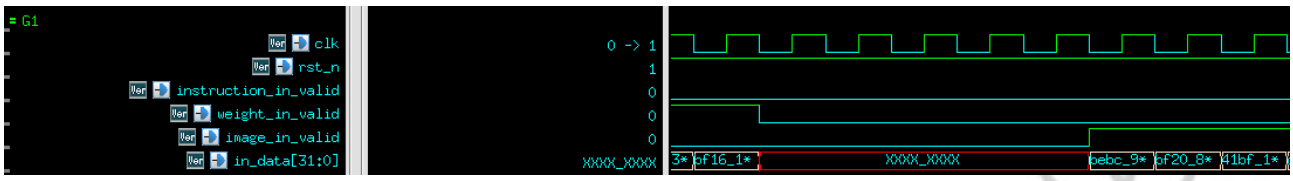


Fig3. Input waveform



Fig4. Output waveform



Fig5. Output waveform